

MACHINE REPRESENTATION OF PROGRAMS

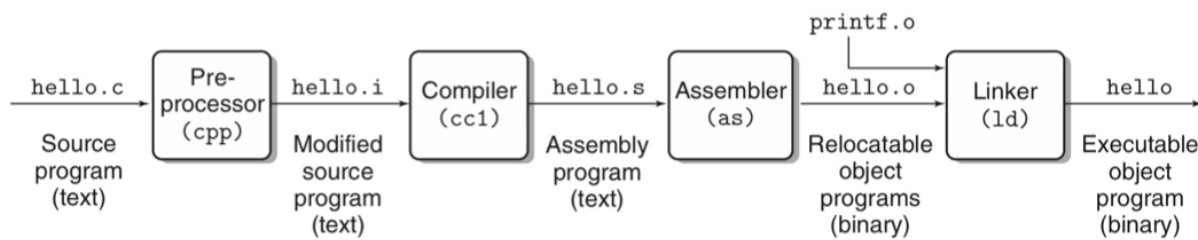
Computers execute machine code

↳ sequence of bytes encoding low level operations

A compiler generates machine code by-

- Rules of the programming language
- Instruction set of the target machine
- Conventions of the Operating System

→ The Compilation Process



Compiler Optimizations can rearrange execution order, eliminate unwanted computations, replace slow operations and change recursive computation to iterative ones.

Moore's Law - The number of transistors per chip will double every year.

↳ Moore's Law has sadly ended. I believe Physics caught up.

The format and behavior of machine level programs is defined by the instruction set architecture. Most ISAs describe the behavior of the program as if each instruction is executed in sequence, with one instruction completing before the next one begins.

High level languages provide a model in which objects of different data types can be declared and allocated in memory, machine code views the memory as simply a large byte-addressable array.

Program memory is addressed using virtual addresses. The OS manages the virtual address space, translating virtual addresses into the physical addresses.

To view the assembly code generated by the C compiler-

> gcc -Og -S hello.c

-Og corresponds to the level of compiler optimization

Disassemblers allow us to inspect contents of the machine level code. They generate a format similar to assembly code from machine code.

x86-64 instructions can range in length from 1 to 15 bytes. Commonly used instructions and those with fewer operands require a smaller number of bytes.

Most assembly code instructions generated by GCC have a single character suffix denoting the size of the operand. Floating point code involves an entirely different set of instructions and registers.

An x86-64 CPU contains a set of 16 general purpose registers storing 64-bit values. These registers are used to store integer data as well as pointers -

%rax → Return Value

%rbx → Caller Saved

%rcx → 4th Argument

%rdx → 3rd Argument

%rsi → 2nd Argument

%rdi → 1st Argument

%rbp → Caller Saved

%rsp → Stack Pointer

%r8 → 5th Argument

%r9 → 6th Argument

%r10 → Caller Saved

%r11 → Caller Saved

%r12 → Caller Saved

%r13 → Caller Saved

%r14 → Caller Saved

%r15 → Caller Saved

→ What happens if we have more than 6 arguments?

• The excess arguments are placed on the stack. Note that accessing the stack is slower than registers.

Assembly Instructions - Instruction Operand [Operand]

↳ Operands are generally source & destination locations

constants, registers, memory ←

↳ registers & memory

→ Data Movement Instructions -

Mov S, D → This instruction copies data from a source location to a destination location without any transformation.

↳ x86 restricts both operands from referencing memory locations. This is because CPU buses are designed to only make one memory reference at a time. Copying a value from one memory location to another requires two instructions: the first to load the source value into a register and the second to write this register to the destination.

MOVZ S, R → Copies a smaller value to a larger destination.

↳ Fills the remaining bytes of the destination with zeroes.

MOVS S, R → Copies a smaller value to a larger destination.

↳ Fills the remaining bytes of the destination with sign extension (replicating copies of the MSB of the source operand).

NOTE: For most Mov instructions, only the specific register bytes mentioned by the operand are updated. However, when we have a 32 bit value to a 64 bit destination, the upper 32 bits are zeroed out.

NOTE: Pointers in C are simply addresses. Dereferencing a pointer involves copying that pointer into a register, and then using this register in a memory access.

NOTE: Local variables are stored in registers as opposed to memory. Register access is faster than memory access.

→ The Stack -

A data structure where data is added in a last in first out manner. It plays a very important role in procedure calls.

↳ In x86, the stack grows downwards such that the top element always has the lowest address of all stack elements. The stack pointer `%rsp` holds the address of the top element.

<u>Pushing</u>		<u>Popping</u>	
<code>subq \$8, %rsp</code>	} <code>pushq %rbp</code>	<code>movq (%rsp), %rax</code>	} <code>popq %rax</code>
<code>movq %rbp, (%rsp)</code>		<code>addq \$8, %rsp</code>	

Since the stack is contained in the same memory as the program code and other forms of program data, programs can access random positions within the stack using standard memory addressing methods.

→ Arithmetic and Logical Operations -

<code>lq S,D: D ← &S</code>	<code>xor S,D: D ← S ^ D</code>
<code>inc D: D ← D + 1</code>	<code>or S,D: D ← D S</code>
<code>dec D: D ← D - 1</code>	<code>and S,D: D ← D & S</code>
<code>neg D: D ← -D</code>	<code>sll R,D: D ← D << R</code>
<code>not D: D ← ~D</code>	<code>sar R,D: D ← D >>_A R → Arithmetic</code>
<code>add S,D: D ← S + D</code>	<code>shr R,D: D ← D >>_L R → Logical</code>
<code>sub S,D: D ← D - S</code>	
<code>imul S,D: D ← D * S</code>	

As with the `mov` instructions, operands to binary instructions cannot both be memory addresses owing to the fact that CPU buses cannot have two memory accesses simultaneously.

→ Control Instructions -

Machine code provides two basic low level mechanisms for implementing conditional behavior - it tests data values and either alters the data or the control flow based on these tests. Data dependent control flow is the more general approach for implementing conditional behaviors.

Generally instructions in both C and machine code are instructed sequentially, i.e., in the order in which they appear in the program.

↳ The execution order can be altered with the jump instruction. This indicates that control should pass to some other part of the program.

The CPU maintains a set of single bit condition code registers describing attributes of the most recent arithmetic or logical operation.

- CF - Carry Flag → The CMP behaves as the SUB instruction
 - ZF - Zero Flag → The TEST behaves as the AND instruction
 - SF - Sign Flag
 - OF - Overflow Flag
- ↳ Both these instructions do not update the destination operands. They only update the condition codes.

Condition codes are not accessed directly. Instead they are used indirectly-

- SET - Set a single byte to 0 or 1 based on some condition.
- JMP - Conditionally jump to some other part of the program.
- CMOV - Conditionally transfer data

→ Jump Instructions -

Under normal execution, instructions follow each other in the order they are listed. A jump instruction can cause the execution to switch to a new position in the program. Jump destinations are generally indicated by a label.

While generating object code file, the assembler determines the address of all labeled instructions and encodes the jump targets as part of the instruction.

Jump instructions can be either conditional or unconditional. Jump targets can either be -

↳ Conditional jumps can only be direct.

1. Direct - the target is encoded as part of the instruction.
2. Indirect - the target is either a register or a memory location.

↳ Direct jumps are written by giving a label as a target → `jmp .L1`

↳ Indirect jumps are written using a * followed by a location → `jmp *(%rax)`

In assembly code, jump targets are written using symbolic labels. The assembler and later the linker generate the proper encodings of the jump targets.

Jump encodings are generally PC relative. PC → Program Counter

↳ They encode difference between the address of the target instruction and the instruction immediately following the jump.

↳ The value of the PC is the address of the instruction following the jump. This is done for historical reasons where the processor would update the PC as the first step in executing an instruction

↳ By using a PC relative encoding of jump targets, the instructions can be compactly encoded, and the object code can be shifted to different positions in memory without alteration.

Processors achieve high performance through pipelining, where an instruction is processed via a series of stages, each performing one small portion of the operation. This approach achieves high performance by overlapping the steps of successive instructions. To do this requires knowledge of the order of instruction execution ahead of time.

↳ This cannot happen reliably when a processor encounters a conditional jump. Processors apply sophisticated branch prediction algorithms to help.

If branch prediction fails, the program incurs significant penalties. This can be avoided by using conditional moves as opposed to conditional jumps. The processor simply reads the source value, checks the condition code, and then either updates the destination or keeps it the same.

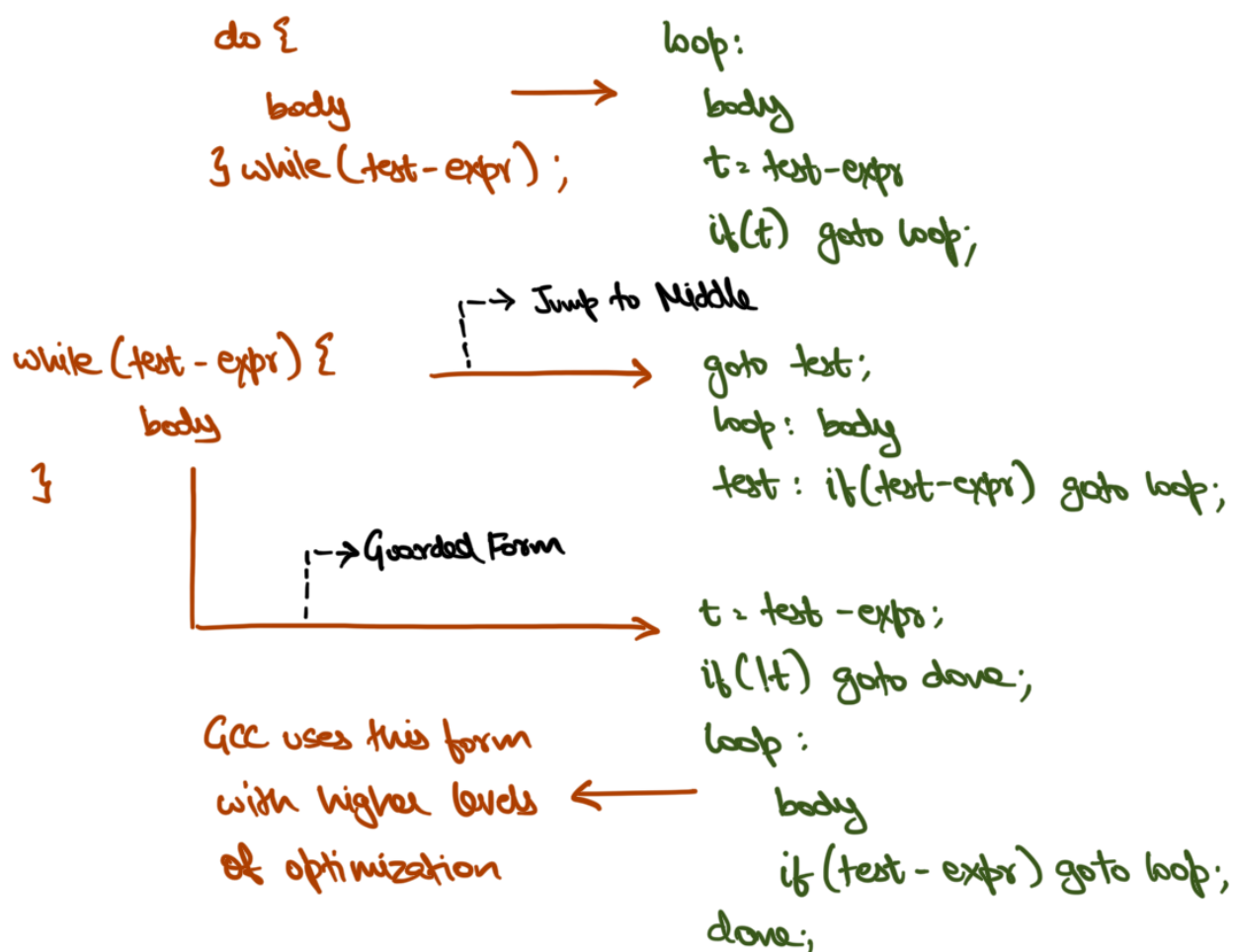
↳ A conditional move in C is the ternary operator -
 $v = \text{test-expr} ? \text{then-expr} : \text{else-expr}$

Conditional moves are efficient as they evaluate the result hands before the conditional assign. However, if either of the hands is computationally intensive, conditional moves would perform worst. It is also the reason why we can't use them for pointer dereferencing -

$p ? *p : 0 \rightarrow *p$ and 0 are evaluated first before checking if p is a non NULL pointer.

→ Loops -

C has many looping constructs - do while, while & for. There is no machine code equivalent. ISA's use a combination of conditional tests and jumps to implement loops.



The C language has the following translation specified for 'for' loops and 'while' loops-

<pre> for (init - expr ; test - expr ; update - expr) { body; } </pre>	→	<pre> init - expr ; while (test - expr) { body; update - expr; } </pre>
--	----------	---

Since for loops are essentially while loops, the compiler will either use "jump to middle" or "guarded to" translation formats based on compiler optimization levels.

→ Switch -

Switch Statements make C code more readable (I hate if statements lol). Switch statements make code more efficient through the jump table.

↳ A jump table is an array where entry i is the address of a code segment to be executed for index $z = i$.

The time taken to do a switch is independent of the number of states. This is not true for if-else blocks. Switch statements generally use indirect jumps, i.e., the jump operand specifies a memory location.

→ Procedures -

Well defined software uses procedures as an abstraction mechanism. A key feature of calling procedures is the use of the run time stack.

↳ The stack and the program registers store information required to pass control and data.

↳ The x86-64 stack grows towards lower addresses.

↳ The stack pointer `%rsp` points to the top element of the stack.

↳ We can add to and remove from the stack using `pushq` and `popq`.



→ The stack frames for most procedures are of a fixed size, allocated at the beginning of the procedure.

→ P calls Q: P can store up to 6 arguments on the stack via registers. All other arguments need to be part of the stack frame.

It is possible that some procedures do not have stack frames. This can happen if all the local variables can be held in registers and the function does not call other functions.

The call instruction is used to transfer control between functions. Transfer of control is done by updating the Program Counter ($\cdot\cdot\cdot$ rip).

↳ Like the jump instruction, call instructions can be direct or indirect.

↳ The ret instruction is used to return from the procedure.

The set of registers is a global resource shared by all procedures. We thus need to ensure that the callee does not overwrite registers the caller planned on using later.

↳ **Callee Saved:** $\cdot\cdot\cdot$ rbp, $\cdot\cdot\cdot$ rbx, $\cdot\cdot\cdot$ r12, $\cdot\cdot\cdot$ r13, $\cdot\cdot\cdot$ r14, $\cdot\cdot\cdot$ r15

The callee ensures that these registers are not modified when it returns.

Can be done by either not modifying the registers or by pushing the original values to stack and restoring before the return.

All other registers except the stack pointer can be modified by any function.

Since each procedure has its own space on the stack, local variables of multiple outstanding calls do not interfere with one another. This allows procedures to call themselves recursively.

→ Array Allocation and Access -

One unusual feature of C is that we can generate pointers to elements within arrays and perform arithmetic with these pointers. These are converted into address computation in machine code. The memory referencing instructions in x86-64 is optimized for arrays.

C allows arithmetic on pointers, where the computed value is scaled according to the size of the data type referenced by the pointer. The array reference $A[i]$ is identical to the expression $*(A+i)$.

→ Heterogeneous Data Structures -

C provides two primary data structures to group unrelated data -

1. Structures → Aggregate related entities into one unit
2. Unions → Allow an object to be referenced using different data types.

→ Structures -

- The different components of a structure are referenced by name.
- All components of a structure are stored in a continuous region of memory.
- The pointer to the structure is the address of the first byte.
- The compiler tracks type information by keeping track of the field offset.
 - ↳ These offsets are used to reference structure elements.
- Structure element selection is done at compile time.
 - ↳ Machine code contains no information about structure elements.

→ Unions -

- Unions provide a way to circumvent C's type system.
 - ↳ A single object can be referenced using multiple types.
 - ↳ All fields in a union reference the same memory location.
- The overall size of an union is the size of the largest element.
- Unions bypass the C type system making them dangerous to use.

→ Data Alignment -

- Processors place restrictions on allowable addresses for primitive data types.
 - ↳ They require addresses to be multiples of some value K (typically 2, 4 or 8)
- This alignment requirement leads to simpler hardware design.
 - ↳ While hardware will work correctly if data is not aligned, but the software would be performant.
- For structures, the compiler needs to insert gaps between fields to maintain alignment.
 - ↳ Compilers may also add padding to the end of structures to maintain alignment.

→ Pointers -

Every pointer has an associated type. The type indicates the type of the object pointed to.

↳ The `void*` represents a generic pointer. They are generally cast to other types.

↳ Pointers are not part of machine code, they are an abstraction provided by C.

Every pointer has a value. Pointers without values point to NULL. In C, pointers are created with `&` and dereferenced with `*`. Casting from one type to another changes the type, but not the value.

Note: Pointers can also point to functions. They are used to pass around code as references (callbacks). The value of a function pointer is the address of the first instruction of the function.

→ Out of Bounds Memory References -

C does not perform bounds checking for array references. We also store local variables on the stack along with state information such as saved register values and return addresses.

↳ This can lead to serious errors where the state stored on the stack is corrupted by an out-of-bounds memory update → common case in buffer overflow

An evil use of Buffer Overflow is to get a program to perform a function it is unwilling to do. ↳ Program is fed a string that has the exploit code and a return instruction to the exploit code.

Worms - A program that can run by itself and propagate fully working versions to other machines.

Viruses - A piece of code that can add itself to other programs. It cannot run independently, and thus needs to be invoked.

→ Preventing Buffer Overflows -

1. Stack Randomization - To insert exploit code, attacker needs to insert code & the pointer to the exploit code. Generating a pointer requires knowing the stack address. The idea of stack randomization is to make the position of the stack vary from one run of program to another.

↳ This can be overcome by an attacker by installing a long list of "noop" operations before the start of the exploit code.

2. Stack Corruption Detection - Modern day compilers, insert a guard value between the local buffer and the rest of the stack. This value is randomly generated everytime a program runs. Before restoring register state and returning from the function, the program checks if the guard value has been altered. If yes, the program exits with an error.



↳ Generally the guard value is stored in a read only segment so the attacker can't modify it. The performance penalty is minimal.

3. Limiting Executable Code Regions - Historically x86 machines merged read access and executable permissions into one. Since the stack had read access by default, all stack addresses could be executed as well.

↳ Later on AMD & Intel added the NX bit to indicate memory locations that cannot be executed. Since this is done in hardware, it has no real performance penalty.